Representing data pipelines graphically

In this train, we'll learn how to represent data pipelines in a graphical format, focusing on visualising workflows using DAGs.

Learning objective

> - Be able to represent data pipelines in a graphical format.

Why Graphical Representation Matters

Graphical representation of data pipelines provides:

- **Clarity:** Easy visualization of data flow.
- **Debugging Aid:** Helps identify bottlenecks and failures.
- **Dependency Mapping:** Displays task relationships.
- **Collaboration Tool:** Assists teams in understanding workflow.

Graphical Representation in Apache Airflow

There are two key concepts used when representing data pipelines as graphs:

- Tasks are represented as graph nodes
- Task dependencies are represented by directed connections (known as edges) between the task nodes

A simple example of representing a data pipeline as a graph can be seen below:

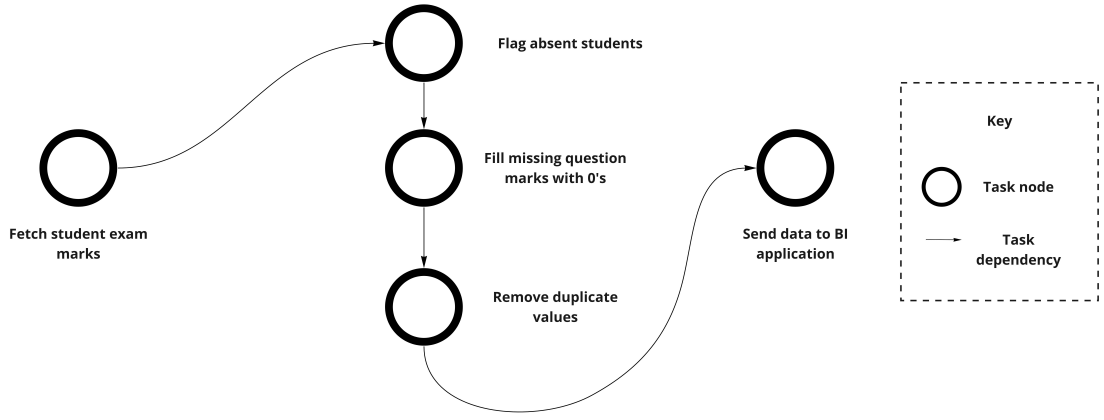


Figure 1: Representing an example data pipeline as a graph.

From Figure 3, the following is clear:

- The first task in the pipeline is to fetch the student marks data.
- Similarly, the last task in the pipeline is to push the transformed data to the BI application.
- The direction of the edges indicates the direction of task dependency within the pipeline. For example, the arrow pointing from the "Flag absent students" node to the "Fill missing question marks with zeroes" node indicates that the flagging operation executes before the filling one.

This type of graph, where there is directional dependence, is referred to as a directed graph. When such a graph contains no loops or cycles (no node has a dependence on nodes further down the chain of execution), it is known as a directed acyclic graph (DAG). To illustrate this point, we can observe in our example graph how task dependence flows from the "Fetch student exam marks" node all the way to the "Send data to BI application" node, without ever referring to a previous task. As such, our example pipeline graph is a DAG.

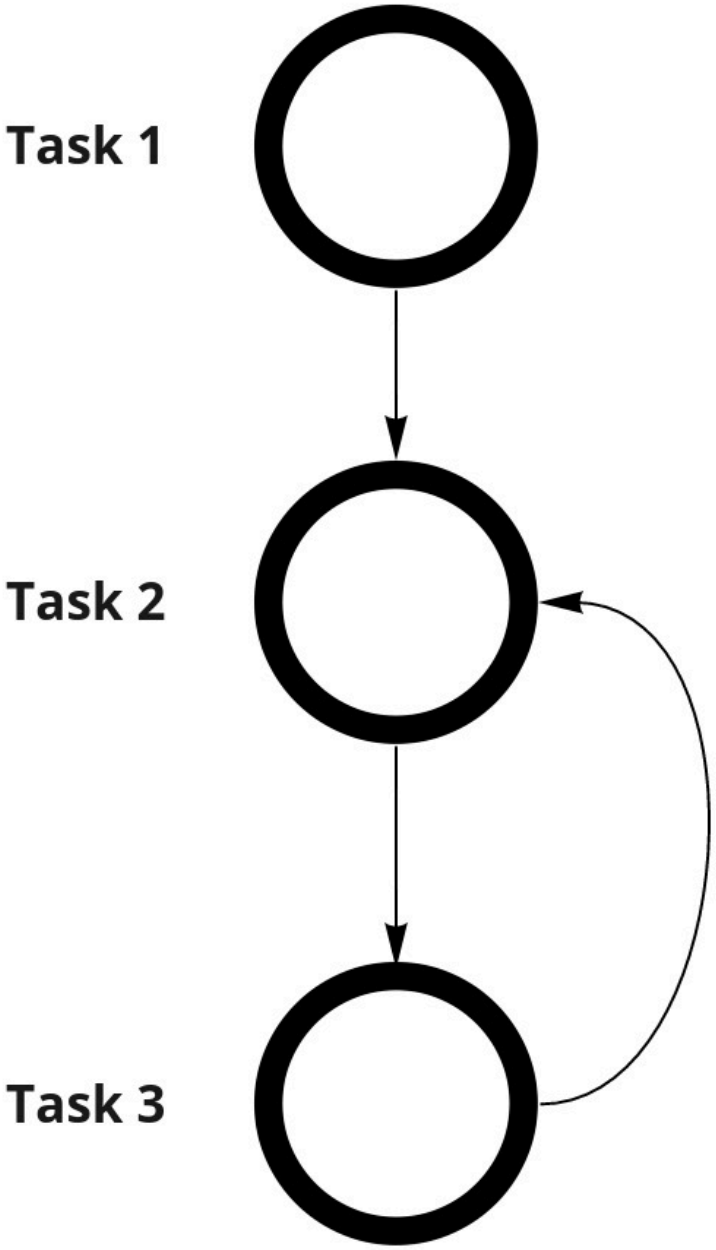


Figure 2: A simple cyclic graph representing a malformed pipeline. Task 2's dependence on Task 3 means that it can never successfully execute, leaving the pipeline in a deadlock.

Let's imagine for a brief moment that there was no requirement for a pipeline, represented by a graph, to be acyclic. How would this affect the pipeline's execution? Figure 2 can help us reason this out. Here we can see directed connections from Task 1 to Task 2, and then to Task 3. However, we also see that Task 3 links back to its previous task. Following this chain of dependence, once Task 1 completes, we expect Task 2 to begin. However, Task 2 can't begin as it awaits output from Task 3, which will never occur as Task 3 needs the output from Task 2. Such a scenario leads to a flaw in our simple example's execution logic, and hence, a pipeline with a cyclic dependence will never execute, as it will remain in a deadlock state.

Conclusion

Representing data pipelines graphically enhances comprehension and troubleshooting. Tools like Apache Airflow enable effective visualization of workflows. By following best practices, teams can create intuitive and maintainable pipeline representations.

Key Takeaways

- **Graphical Representation Aids Understanding:** Visualizing data pipelines using Directed Acyclic Graphs (DAGs) helps clarify workflow structure, making it easier to identify dependencies, bottlenecks, and execution order.
- **DAGs Ensure Proper Execution:** Acyclic graphs prevent deadlocks by ensuring tasks progress in a linear, dependency-driven manner. Cyclic dependencies can cause execution failures due to tasks waiting on each other indefinitely.
- **Apache Airflow as a Visualization Tool:** Apache Airflow provides a powerful way to represent, schedule, and monitor data pipelines graphically, improving debugging, collaboration, and pipeline maintainability.